

A Hybrid Knowledge-Graph RAG Pipeline for Legal PDF Corpora: Integrating ODKE+ Extraction with KG-GPT and ARK-V1 Reasoning

Samuel Bagin

April 16, 2026

Abstract

We present the design and implementation of **PDFGraphRAG**, a Python system that transforms Slovak-language legal PDFs into a Neo4j knowledge graph and answers natural-language questions over it. The ingestion stage follows the *Open-Domain Detection* $\rightarrow$ *Schema Refinement* $\rightarrow$ *Schema-Driven Extraction* (ODD/SR/SDE) paradigm popularised by ODKE+ [1], augmented with a document-layout-aware table extractor based on DocLayout-YOLO. The query stage follows the three-stage KG-GPT pipeline [2]—sentence segmentation, graph retrieval and inference—but replaces the original LLM ranker of Stage 2 with the ARK-V1 [3] state-machine retriever, which grounds every relation selection in the live graph schema. This document formalises the pipeline, describes each module of `code/pdf_graphrag.py`, and relates the design choices to the three source papers using mathematical notation.

1 Introduction

Retrieval-Augmented Generation (RAG) over unstructured text is now standard, yet vector similarity alone collapses the rich relational structure implicit in legal documents: paragraphs, sections, obligations and temporal constraints. Knowledge-graph RAG (KG-RAG) re-introduces that structure by grounding an LLM’s answers in a typed graph $G = (V, E, \mathcal{T}_V, \mathcal{T}_E)$ with node labels $\mathcal{T}_V$ and relation labels $\mathcal{T}_E$. Our system combines three recent lines of work:

1. **ODKE+** [1], which advocates an ontology-guided extraction loop with a distinct *grounder* that filters hallucinations, reporting a 35% reduction in unsupported facts.
2. **KG-GPT** [2], which decomposes KG question answering into (SEGMENT, RETRIEVE, INFER).
3. **ARK-V1** [3], a state-machine agent that iterates (RELATION-ENUM, RELATION-SELECT, TRIPLE-ENUM, REASON) steps up to a bounded depth $K_{\max}$.

Section 2 describes the ingestion phase, Section 3 the query phase, Section 4 discusses implementation details, and Section 5 places the design against the cited literature.

2 Ingestion: From PDF to Typed Graph

Let a document D be a sequence of pages $(p_1, \dots, p_n)$. The ingestion pipeline produces two artefacts: a graph $G_D \subseteq G$ and a set of vector indices $\{\mathbf{V}_N, \mathbf{V}_C, \mathbf{V}_R\}$ over nodes, chunks and relation-type labels respectively. The top-level driver is `PDFGraphRAG.process` in `code/pdf_graphrag.py`.

2.1 Layout-aware table extraction

Before textual processing, `detect_tables` rasterises the PDF at 200 DPI and runs a DocLayout-YOLO v10 model (checkpoint `juliozhao/DocLayout-YOLO-DocStructBench`) with a confidence threshold $\tau = 0.3$. For each page p_i the detector returns a set of bounding boxes $B_i = \{(x_1, y_1, x_2, y_2, c)\}$ where $c \in \mathcal{C}_{\text{target}} = \{\text{table, picture, figure, isolate_formula, formula_caption}\}$. Consecutive table detections (page distance ≤ 1) are grouped by `group_table_detections` into multi-page tables. Each group is sent to `transform_table_to_html`, which prompts a vision-enabled Gemini 2.5 Pro model with the concatenated page crops and a strict structured-output schema `TableHTMLResponse = (html, page_range, row_count, column_count)`. The resulting HTML is then transformed into a hierarchical subgraph by `transform_html_to_graph_document`, which instructs the LLM to emit a rooted tree whose nodes satisfy the invariants

$$\forall v \in V_{\text{table}} : \text{id}(v) \text{ unique} \wedge \text{name} \in \text{props}(v) \wedge \text{ASCII-normalised}(v).$$

Pages that are strictly interior to a multi-page table are removed from the text stream to avoid duplicate extraction, while the first and last pages are retained because they may contain surrounding prose.

2.2 Open-Domain Detection (ODD)

ODD mirrors the “schema discovery” phase of ODKE+ [1], in which a language model proposes candidate predicates before ontology pruning. The text is split with a `RecursiveCharacterTextSplitter` of chunk size $s = 1200$ and overlap $o = 200$. For each chunk c_i an asynchronous agent (`open_domain_detection`) returns a local schema

$$S_i = (\mathcal{T}_V^{(i)}, \mathcal{T}_E^{(i)}),$$

by calling `create_agent` with a `ProviderStrategy` that forces adherence to `response_schema_for_odd`. Concurrency is controlled by an `asyncio.Semaphore(M)` (default $M = 5$) and a cooperative pause/retry event that back-offs the whole batch for 60s on provider errors, matching the robustness pattern recommended by ODKE+ for production throughput [1]. The aggregated schema is

$$S_{\text{ODD}} = \left(\bigcup_i \mathcal{T}_V^{(i)}, \bigcup_i \mathcal{T}_E^{(i)} \right).$$

2.3 Schema Refinement (SR)

The aggregated schema is still noisy: diacritic variants (e.g., *Dan* vs. *Dañ*), near-synonyms and ontology mismatches are common. `schema_refinement` invokes Gemini 2.5 Pro with a structured-output schema `SchemaRefinementResponse` that returns canonical label sets together with a merge log $\mathcal{M} : S_{\text{ODD}} \rightarrow S^*$. The normalisation is conservative—the prompt explicitly forbids merging ontologically distinct kinds (person vs. document) and enforces `UPPER_SNAKE_CASE` for relations. Following ODKE+ [1], an *existing ontology* S_0 (the current Neo4j schema) is passed in so that the refined schema S^* remains compatible with facts already stored:

$$S^* = \Phi(S_{\text{ODD}}, S_0), \quad \Phi = \text{LLM-refiner}.$$

2.4 Schema-Driven Extraction (SDE)

With S^* fixed, the text is resplit with $s = 1024$, $o = 128$ and processed by `async_schema_driven_extraction`. Each chunk c_i yields a `GraphDocument` $G_i = (V_i, E_i, \text{src}_i)$ via `_convert_to_graph_document`. This converter enforces four invariants that together implement an in-pipeline *grounder* analogous to the one advocated by ODKE+ [1]:

1. **ID validation:** nodes without a non-empty `id` are dropped (line 441 of `pdf_graphrag.py`).
2. **Type fallback:** `format_node_type` defaults empty labels to `DEFAULT_NODE_TYPE = Entity`.
3. **Property keying:** `format_property_key` converts keys to camelCase; `format_relationship_type` normalises relations to uppercase-underscore.
4. **Relationship grounding:** a relation is emitted only if both endpoints resolve to a node of the same chunk (case-insensitive match), and a **Chunk** node is linked to every extracted node via `IN_CHUNK`.

An optional `_filter_by_strict_mode` post-filters the graph to the allowed label sets, preserving **Chunk** nodes and `IN_CHUNK` edges by construction.

2.5 Persistence and vector stores

The graph documents—both from text (SDE) and from tables—are inserted via `Neo4jGraph.add_graph_documents` with `baseEntityLabel=True`. Two Cypher statements then strip the auxiliary label from **Chunk** and **Document** nodes so that the entity vector index excludes them. Three `Neo4jVector` indices are built with OpenAI `text-embedding-3-large`:

$\mathbf{V}_N$ over nodes, $\mathbf{V}_C$ over chunks, $\mathbf{V}_R$ over relation-type labels.

The relation-type index is populated by querying `CALL db.relationshipTypes()` and embedding each label as its own pseudo-document, enabling later relation-matching at query time.

3 Query: Three-Stage KG-GPT with ARK-V1 Retrieval

Given a question q , `PDFGraphRAG.query` implements the canonical KG-GPT pipeline [2]

$$q \xrightarrow{\text{Stage 1}} \{s_j\}_{j=1}^m \xrightarrow{\text{Stage 2}} \mathcal{E} \xrightarrow{\text{Stage 3}} a,$$

where s_j are sub-sentences, $\mathcal{E}$ is a set of evidence triples and a is a natural-language answer.

3.1 Stage 1: Sentence segmentation

`segment_question` prompts `openai_client` with `system_prompt_for_segmentation`. The agent emits a `response_schema_for_segmentation` object parsed into `SubSentence` records:

$$s_j = (\text{text}_j, \text{entities}_j), \quad |\text{entities}_j| \leq 2.$$

The constraint of at most two anchor entities per sub-sentence enforces the KG-GPT assumption [2] that every sub-sentence corresponds to a *single implied triple* (h, r, t) . If segmentation fails, the pipeline falls back to a one-element list $\{(q, \emptyset)\}$ to preserve liveness.

3.2 Stage 2: ARK-V1 state-machine retrieval

Whereas KG-GPT’s original Stage 2 scores a static candidate set of relations, our implementation uses the ARK-V1 [3] state-machine, realised by `ark_v1_retrieve`. The state is a 4-tuple

$$\sigma_k = (a_k, \text{summary}_k, R^{(k)}, T^{(k)}),$$

where a_k is the current anchor, summary_k is a running NL summary of prior steps, $R^{(k)} \subseteq \mathcal{T}_E$ are the relations actually present on a_k in the graph, and $T^{(k)}$ is the candidate triple set. A step proceeds as follows:

1. **Anchor resolution** (`_resolve_anchor`): an entity mention is lifted to an actual graph node by case-insensitive exact match, with a substring fallback.
2. **Relation enumeration** (`_retrieve_relations`): $R^{(k)} = \{\text{type}(r) \mid \exists u. (a_k, r, u) \in E \text{ or } (u, r, a_k) \in E\}$.
3. **Relation selection** (`_select_relation`): `gemini_client_flash` with a `RelationChoice` schema picks $r_k \in R^{(k)} \cup \{\perp\}$. Up to $C_{\max} = 2$ retries re-prompt the LLM if it hallucinates a relation outside $R^{(k)}$; this is exactly the *grounded relation selection* invariant of ARK-V1 [3].
4. **Triple enumeration** (`_retrieve_triples`): $T^{(k)} = \{(h, r_k, t) \in E\}$ limited to 25 rows per direction.
5. **Reasoning** (`_reason_step`): an LLM returns a `ReasoningStep` object
$$\rho_k = (I_k \subseteq \{0, \dots, |T^{(k)}| - 1\}, \text{impl}_k, \text{cont}_k \in \{0, 1\}, a_{k+1} \in \{t : (h, r_k, t) \in T^{(k)}\} \cup \{\perp\}).$$
 Selected triples are appended to $\mathcal{E}$ and their endpoints to the node list used later for chunk retrieval.

The loop terminates when (i) $R^{(k)} = \emptyset$, (ii) the LLM emits $\perp$, (iii) $\text{cont}_k = 0$, (iv) the proposed a_{k+1} is not a tail of a selected triple, or (v) the depth cap $k = K_{\max} = 3$ is reached. The final evidence set is de-duplicated while preserving insertion order:

$$\mathcal{E}_{\text{final}} = \text{dedup}\left(\bigcup_j \bigcup_k \{t : t \in T^{(k)}, \text{idx}(t) \in I_k\}\right).$$

Two design details deserve emphasis. First, the prompt for `_select_relation` is reconstructed from scratch every step (system + sub-sentence + summary + current $R^{(k)}$), so the token budget is $\mathcal{O}(1)$ in k rather than growing with the reasoning trace, exactly as recommended by Klein and Ohnemus [3]. Second, every Cypher query passes through `is_read_only_cypher`, which rejects write keywords and the `CALL` procedure family via pre-compiled regular expressions, giving the agent a capability-restricted tool surface.

3.3 Chunk retrieval via IN_CHUNK

After Stage 2, `get_chunks_from_nodes` issues

```
MATCH (n)-[:IN_CHUNK]->(c:Chunk)
WHERE n.id IN $node_ids
RETURN DISTINCT c.text, c.page, n.id
ORDER BY c.page
```

to recover the original text passages from which each retrieved entity was extracted. This corresponds to the KG-GPT [2] observation that symbolic triples alone can be too terse to disambiguate numeric or temporal constraints, and that attaching source text to the prompt substantially improves final-answer faithfulness.

3.4 Stage 3: Inference

`answer` formats the evidence as a JSON list $\mathcal{E}_{\text{final}} = \{[h, r, t]\}$ and the chunks as a `\n--\n`-delimited string, then invokes `gemini_client_flash` with a structured schema `InferenceAnswer` so that the model’s output is forced into a single `answer` field. The prompt explicitly instructs the model to treat the triples as *primary evidence* and the chunks as *supplementary context*, which is the precedence rule proposed in KG-GPT’s Stage-3 ablations [2]. Schematically,

$$a = \text{LLM}(q, \mathcal{E}_{\text{final}}, \mathcal{C}_{\text{final}}),$$

where $\mathcal{C}_{\text{final}}$ is the chunk context.

4 Implementation Notes

4.1 LLM router

The constructor of `PDFGraphRAG` instantiates four clients: an OpenAI model for agentic tasks (segmentation, ODD, SDE, search-agent), two Gemini 2.5 Pro variants (baseline and high-thinking) for heavier reasoning, a Gemini 2.5 Flash for lightweight schema-constrained calls (relation selection, reasoning step, final answer), and an Anthropic client kept for future use. Cost/latency arguments thus drive the routing: Flash for per-step agent calls inside the ARK-V1 loop (latency-sensitive, small context), Pro for one-shot refinement and table-to-graph extraction (higher accuracy at larger context).

4.2 Serialisation

`serialize_for_json` recognises Neo4j `Node`, `Relationship` and `Path` objects by duck-typing (`hasattr(obj, 'labels')` etc.) and recursively converts them to JSON-safe dictionaries. This is required because the `search_database` tool inside `search_agent_retrieve` returns native driver objects that the Langchain tool contract must stringify.

4.3 Idempotent graph writes

`add_graph_docs_without_apoc` issues `MERGE` on both nodes and relationships, so re-ingesting the same PDF does not duplicate edges. The main `process` method prefers `graph.add_graph_documents` with `baseEntityLabel=True`, which depends on APOC and is therefore a deployment constraint for the target Neo4j instance.

5 Discussion

Relation to ODKE+. ODKE+ [1] targets Wikipedia-scale fact freshness with a five-stage pipeline (Initiator, Retriever, Extractor, Grounder, Corroborator) and reports 98.8% precision after corroboration. Our ingestion adopts the *Extractor* and *Grounder* abstractions: the ODD/S-R/SDE triple is a specialisation of ODKE+’s ontology-guided extractor for the case where the ontology is only *partially* known (we bootstrap it from the corpus), while `_convert_to_graph_document` and `_filter_by_strict_mode` together play the role of the grounder. Unlike ODKE+ we do not yet implement a *corroborator* stage—a natural extension would be cross-chunk agreement scoring before persistence.

Relation to KG-GPT. Kim et al. [2] split KGQA into three LLM-driven stages and show that factoring the problem this way outperforms monolithic prompting on MetaQA and FactKG. We reproduce their factorisation verbatim at the level of the `query` method, and we additionally import their observation that attaching raw text context to Stage 3 improves numeric/temporal reasoning. The novel element is the substitution of Stage 2: KG-GPT’s Lexical-Matching-and-LLM-Ranking (LMLR) retriever is replaced by ARK-V1.

Relation to ARK-V1. Klein and Ohnemus [3] formalise the retriever as a Mealy machine whose transitions are gated by LLM calls constrained to the empirically observed relation set $R^{(k)}$. Our implementation follows the paper closely: bounded depth, bounded retries on invalid LLM outputs, and a constant-size prompt that elides older intermediate steps behind an NL summary. Deviations are minor: we bound anchor expansion to the first two entity mentions (`(entities or [])[:2]`) and the triple limit to 25 per direction—both pragmatic caps that keep Gemini Flash calls below the free-tier token ceiling.

Limitations. (i) The grounder uses surface-form validation only; a semantic grounder (entailment classifier) as in ODKE+ [1] would further reduce hallucinations. (ii) ARK-V1 with $K_{\max} = 3$ under-covers genuine multi-hop questions; adaptive depth selection conditioned on sub-sentence complexity would be an upgrade. (iii) Slovak diacritics are stripped at extraction time; a post-hoc re-diacritisation step would improve user-facing rendering of the final answer.

6 Conclusion

PDFGraphRAG is, to our knowledge, the first open implementation that composes ODKE+-style hybrid extraction [1] with the KG-GPT three-stage query pipeline [2] and the ARK-V1 state-machine retriever [3] in a single Python module, targeted at Slovak-language legal corpora. The ingestion side contributes a layout-aware table extractor that preserves multi-page table hierarchies as typed subgraphs, and the query side contributes an IN_CHUNK-based dual retrieval that feeds both symbolic triples and source prose to the final LLM. Empirical evaluation on a held-out Slovak tax-law benchmark is left to future work.

References

- [1] Khorshidi, S. et al. (Apple Inc.). *ODKE+: Ontology-Driven Knowledge Extraction at Web Scale*. arXiv:2509.04696, 2025. <https://arxiv.org/html/2509.04696v1>.
- [2] Kim, J., Kwon, Y., Jo, Y., and Choi, E. *KG-GPT: A General Framework for Reasoning on Knowledge Graphs Using Large Language Models*. Findings of EMNLP 2023. <https://aclanthology.org/2023.findings-emnlp.631.pdf>.
- [3] Klein, J. F. and Ohnemus, J. *ARK-V1: A State-Machine LLM Agent for Knowledge-Graph Question Answering*. arXiv:2509.18063, 2025. <https://arxiv.org/pdf/2509.18063>.